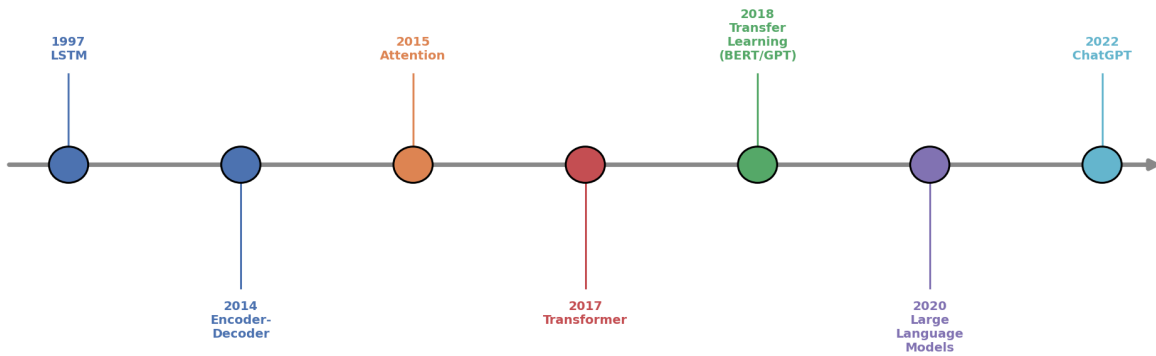


The Epic History of Large Language Models

From LSTMs to ChatGPT - Six Stages That Changed AI

The Epic Journey: From LSTMs to ChatGPT



Stage 1: Encoder-Decoder | Stage 2: Attention | Stage 3: Transformer | Stage 4: Transfer Learning |
Stage 5: LLMs | Stage 6: ChatGPT

The Story So Far

Modern LLMs like ChatGPT didn't appear out of nowhere - they're the result of a decades-long chain of breakthroughs, each one solving the biggest limitation of the previous approach. This guide walks through that chain, stage by stage: from LSTMs struggling with long sequences, to encoder-decoder models, to attention, to the Transformer, to transfer learning, to today's massive LLMs, and finally to ChatGPT itself.

STAGE 1 - Encoder-Decoder Architecture

By the mid-2010s, LSTMs had become the standard tool for sequence tasks, but a new challenge emerged: how do you map one sequence to a completely different sequence, such as translating an English sentence into French? The **encoder-decoder** (or "Seq2Seq") architecture, introduced in 2014, was the answer.

The idea is simple: an **encoder** (typically an LSTM) reads the entire input sequence and compresses it into a single fixed-length **context vector**. A separate **decoder** (also an LSTM) then reads that context vector and generates the output sequence, one token at a time.

Stage 1 - Encoder-Decoder (Seq2Seq) Architecture

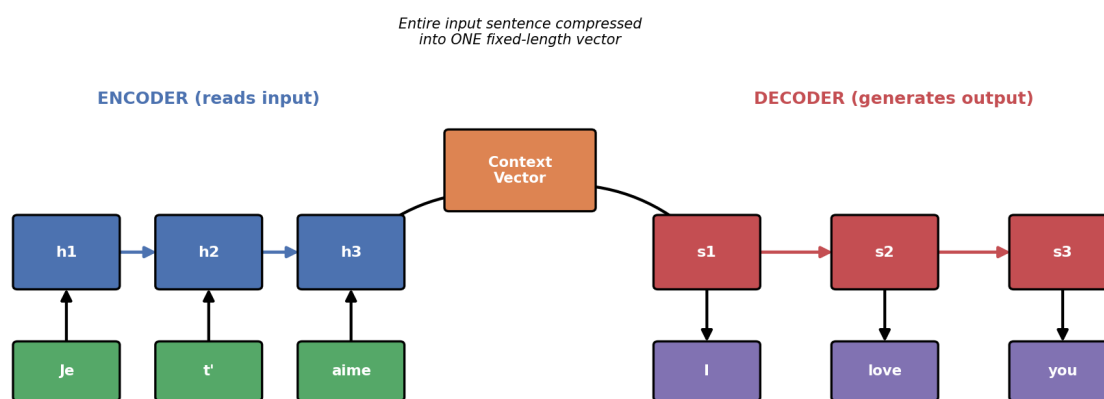
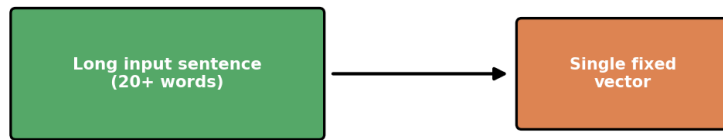


Figure 1: The encoder compresses the source sentence into a context vector; the decoder unpacks it into the target sentence.

The Problem: A Bottleneck

This worked well for short sentences, but broke down on longer ones. Forcing an entire sentence - no matter how long - through a single fixed-size vector meant that information from early words often got lost by the time the decoder needed it.

The Bottleneck Problem



*Squeezing a long sequence into one small vector loses information -
performance degrades badly on long sentences*

Figure 2: The fixed-length context vector is a bottleneck - it can't hold unlimited information as sentences grow longer.

STAGE 2 - The Attention Mechanism

In 2014-2015, Bahdanau et al. introduced **attention**, directly solving the bottleneck problem. Instead of forcing the decoder to rely on a single fixed context vector, attention lets the decoder "look back" at **every** encoder hidden state at every decoding step, and decide which ones are most relevant right now.

Stage 2 - The Attention Mechanism

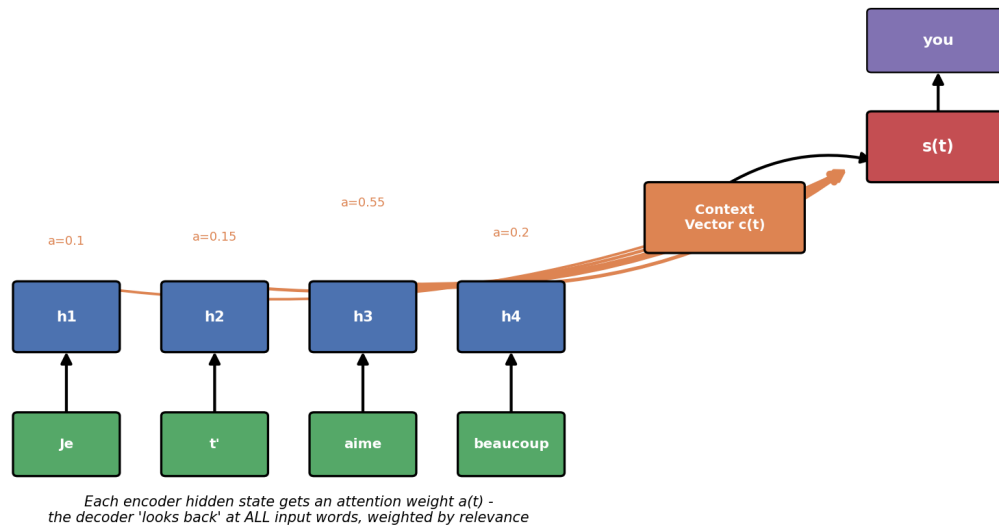


Figure 3: At each decoding step, attention computes a weighted combination of ALL encoder hidden states - not just the last one.

The weights (often written $a(t)$) are learned, and they change dynamically at every decoding step - the decoder essentially asks, "given what I'm generating right now, which input words matter most?" This is often visualized as a heatmap:

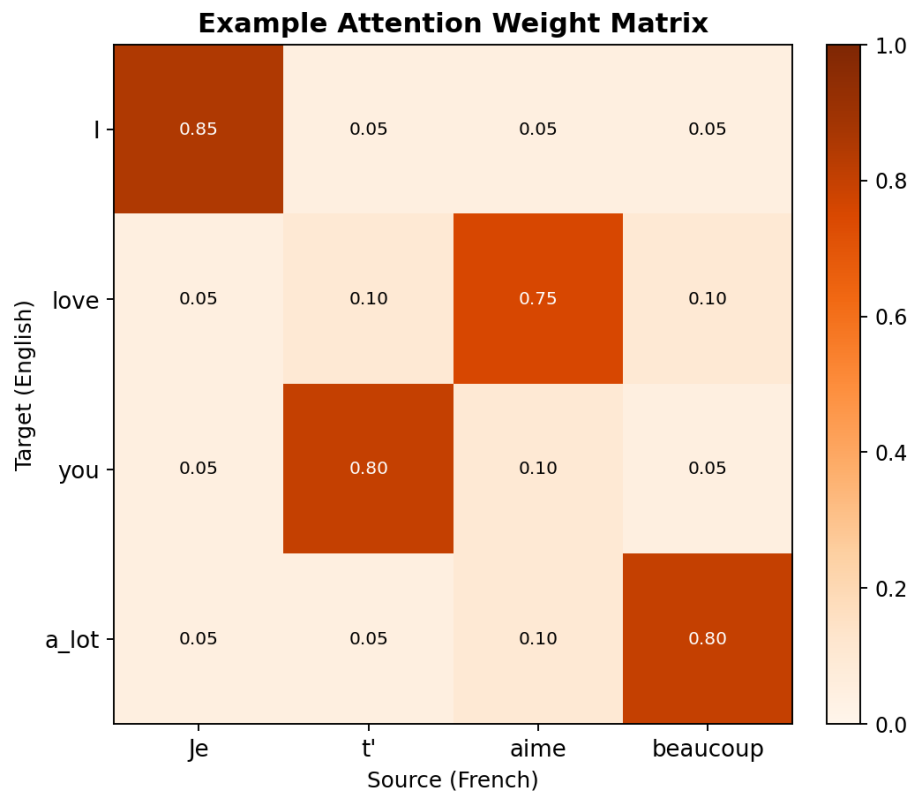


Figure 4: An attention weight matrix - brighter cells mean the target word is paying more attention to that source word.

This single idea - letting a model dynamically focus on the most relevant parts of its input - turned out to be one of the most important breakthroughs in the entire history of deep learning. It set the stage for removing recurrence altogether.

STAGE 3 - The Transformer

In 2017, the paper "*Attention Is All You Need*" asked a bold question: what if we removed recurrence (RNNs/LSTMs) entirely, and built a model using **only attention**? The result was the **Transformer** - and it changed everything.

The key innovation is **self-attention**: instead of only attending from decoder to encoder, every word in a sequence attends to every other word in the same sequence, directly, regardless of distance.

Self-Attention: Every Word Looks at Every Other Word

No recurrence needed - attention connects any two positions directly, regardless of distance, and all positions can be processed in parallel

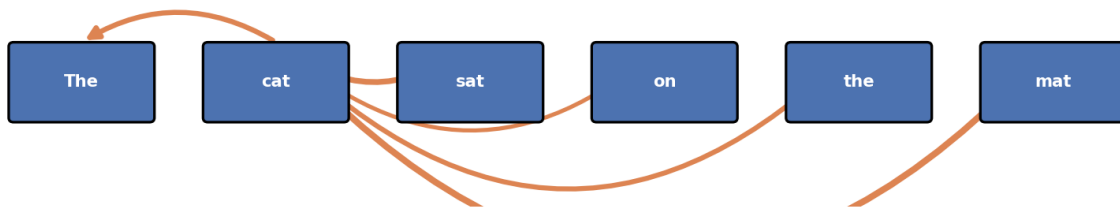


Figure 5: Self-attention lets any word directly relate to any other word, with no step-by-step recurrence required.

Because there's no recurrence, all positions in a sequence can be processed **in parallel** during training - a massive speed advantage over RNNs/LSTMs, which must process one time step at a time. This is a huge part of why Transformers scale so well.

The Full Architecture

The Transformer keeps the encoder-decoder structure from Stage 1, but replaces every recurrent layer with multi-head self-attention and feed-forward layers, plus positional encodings (since there's no recurrence to naturally track word order):

Stage 3 - The Transformer Architecture

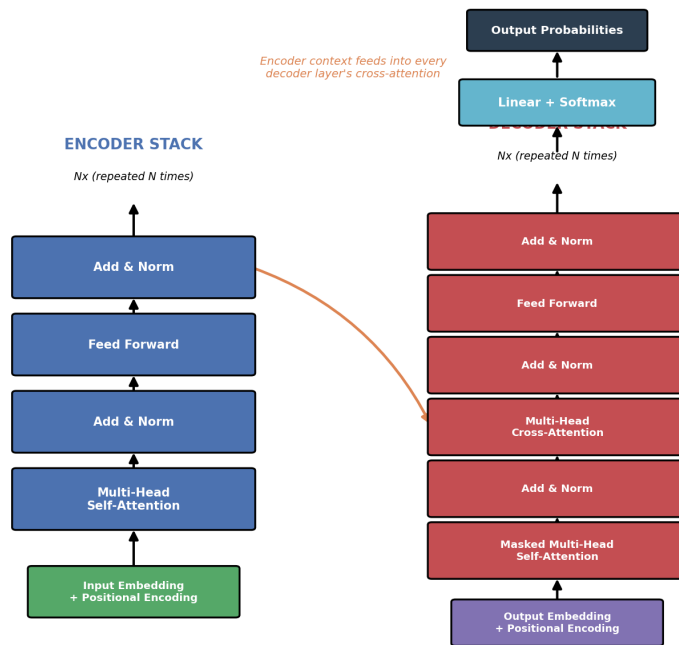


Figure 6: The full Transformer - stacked self-attention and feed-forward blocks in both the encoder and decoder.

STAGE 4 - Transfer Learning

With the Transformer architecture available, researchers in 2018 made another key shift: instead of training a new model from scratch for every task, why not **pretrain** one giant model on massive amounts of unlabeled text, then **fine-tune** it cheaply for many different downstream tasks?

This is the **transfer learning** paradigm, popularized by models like **BERT** (Google, 2018) and **GPT-1** (OpenAI, 2018). Pretraining teaches the model general language understanding; fine-tuning specializes that knowledge for a specific task using a much smaller labeled dataset.

Stage 4 - Transfer Learning: Pretrain Once, Fine-Tune Many Times

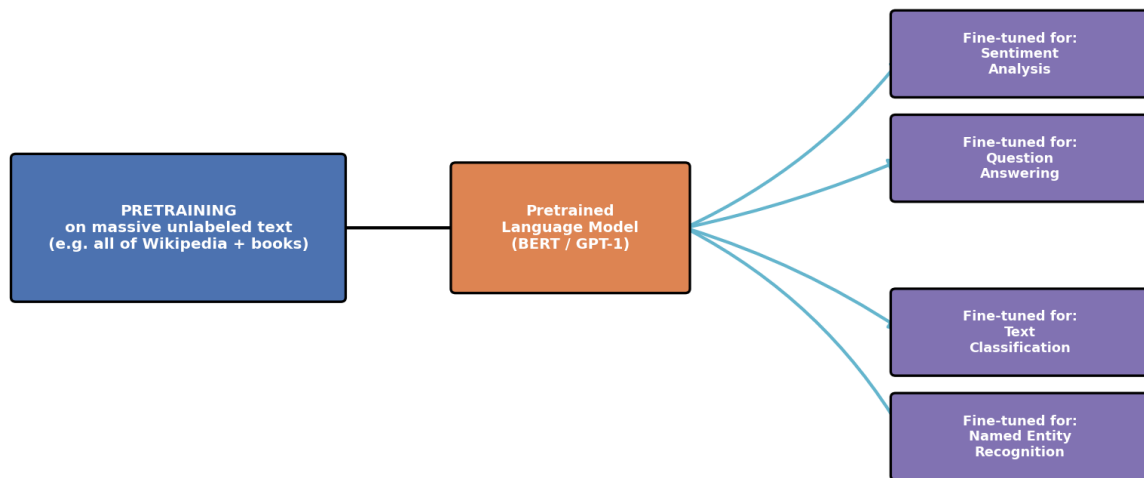


Figure 7: One pretrained model, fine-tuned many different ways for many different downstream tasks.

- **Pretraining** - trained once on huge, general text corpora (Wikipedia, books, web text) using self-supervised objectives (e.g. predicting masked or next words) - no manual labels needed.
- **Fine-tuning** - the pretrained model is further trained on a small, labeled dataset for a specific task, adjusting its weights slightly to specialize.

This dramatically reduced the amount of labeled data and compute needed to get state-of-the-art results on any individual task, since most of the "hard work" of learning language was already done during pretraining.

STAGE 5 - Large Language Models (LLMs)

Once the pretrain-then-finetune recipe was proven, the next question was: what happens if we just make the model - and its training data - much, much bigger? The answer, starting around 2019-2020, was **Large Language Models (LLMs)**: models with tens or hundreds of billions of parameters, trained on huge fractions of the internet.

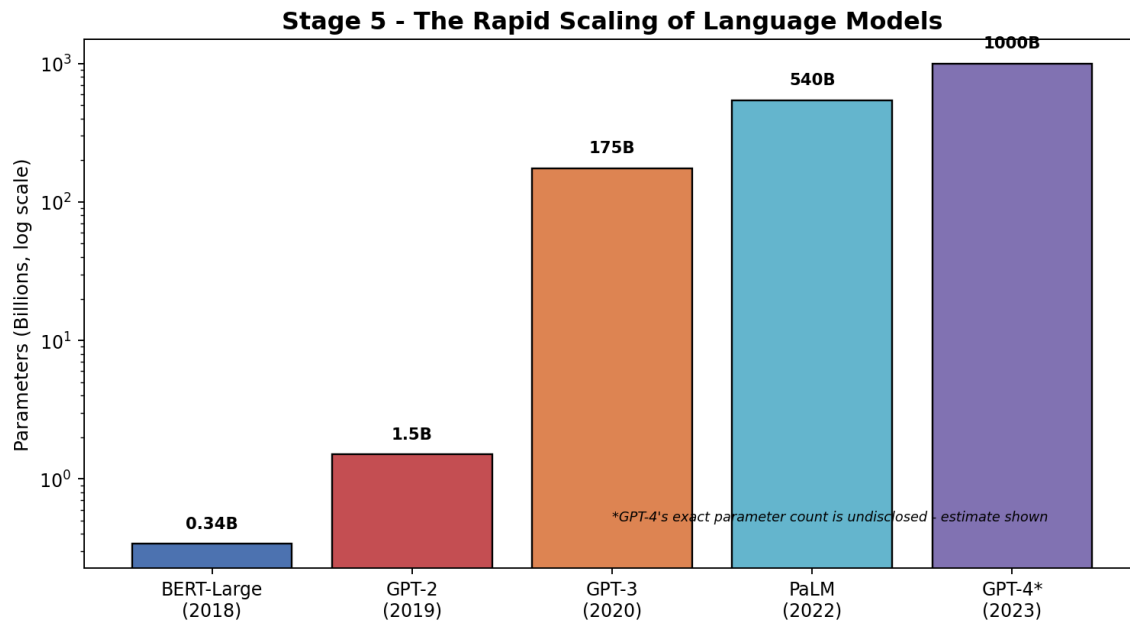


Figure 8: Model sizes grew by roughly 3 orders of magnitude in just a few years.

Scaling up revealed something unexpected: certain abilities - like multi-step reasoning, arithmetic, or following complex instructions - didn't improve gradually. Instead, they seemed to appear suddenly once a model crossed a certain scale threshold. Researchers call these **emergent abilities**.

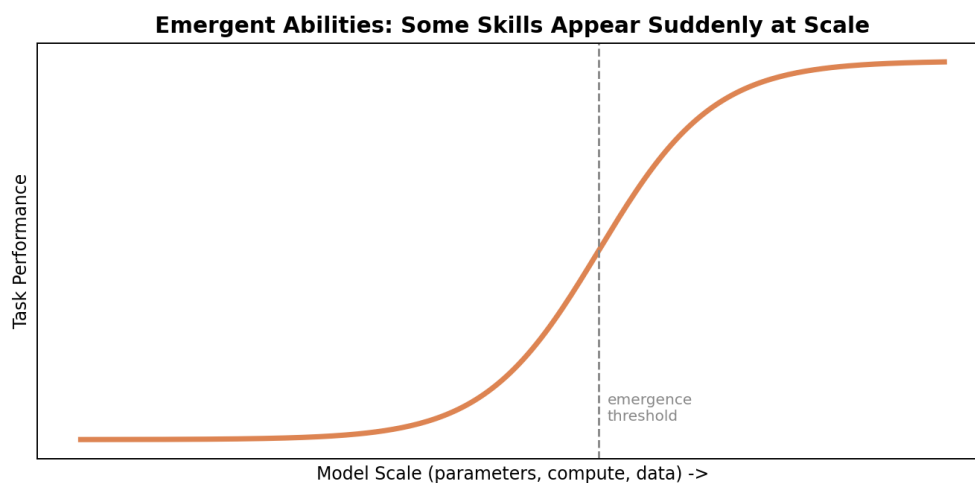


Figure 9: Some capabilities stay near-zero, then rapidly emerge once a model is large enough.

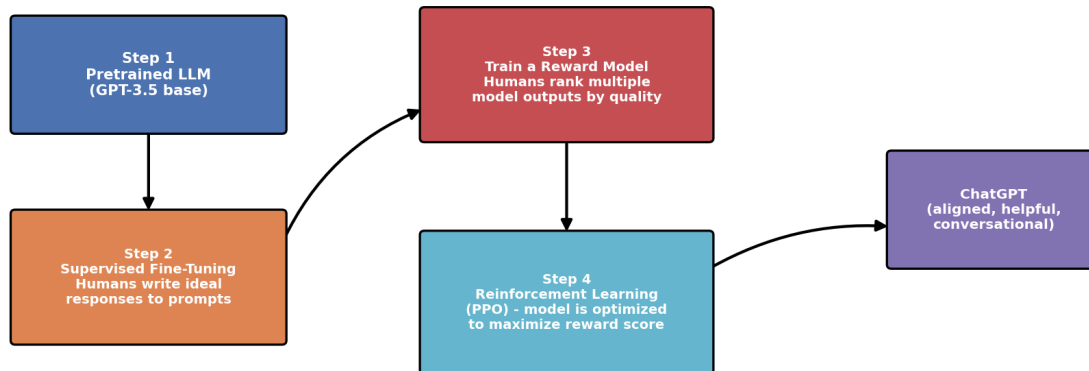
GPT-3 (2020), with 175 billion parameters, was a landmark moment - it showed strong performance on many tasks with just a handful of examples in the prompt ("few-shot learning"), without any task-specific

fine-tuning at all.

STAGE 6 - The Finale: ChatGPT

GPT-3 was powerful, but raw next-word prediction doesn't automatically make a model helpful, safe, or good at following instructions - it just makes it good at continuing text. The final piece of the puzzle, used to build **ChatGPT** (2022), was a training technique called **Reinforcement Learning from Human Feedback (RLHF)**.

Stage 6 - The Finale: How ChatGPT Was Built (RLHF)



This human feedback loop (RLHF) is what makes ChatGPT helpful, safer, and conversational - not just a raw next-word predictor

Figure 10: The RLHF pipeline - from a raw pretrained LLM to a helpful, aligned conversational assistant.

The Four Steps of RLHF

- **1. Start with a pretrained LLM** (GPT-3.5), which already has broad language ability from Stage 5.
- **2. Supervised fine-tuning** - human labelers write high-quality example responses to prompts, and the model is fine-tuned to imitate this style of helpful response.
- **3. Train a reward model** - humans compare multiple model outputs for the same prompt and rank them by quality; a separate model learns to predict these human preference scores.
- **4. Reinforcement learning (PPO)** - the language model is further optimized to generate responses that score highly according to the reward model, using reinforcement learning.

This human-feedback loop is what transforms a raw, powerful-but-unruly text predictor into a model that feels conversational, follows instructions, and (mostly) avoids unhelpful or unsafe responses - which is exactly the experience people recognized as "ChatGPT" when it launched in November 2022.

Summary - The Full Journey

Stage	Year	Key Idea	Problem It Solved
1. Encoder-Decoder	2014	Two RNNs: encode input, decode output	Sequence-to-sequence tasks (e.g. translation)
2. Attention	2014-15	Dynamically weight all encoder states	Fixed-length context vector bottleneck
3. Transformer	2017	Self-attention, no recurrence	Slow, sequential training in RNNs
4. Transfer Learning	2018	Pretrain once, fine-tune for many tasks	Needing huge labeled data per task
5. LLMs	2019-20	Massive scale (100B+ parameters)	Limited generalization & few-shot ability
6. ChatGPT (RLHF)	2022	Human feedback shapes model behavior	Raw LLMs aren't naturally helpful/aligned

Each stage in this story solved the defining limitation of the one before it: encoder-decoder enabled sequence-to-sequence tasks, attention fixed its bottleneck, the Transformer made attention the whole architecture and unlocked parallel training, transfer learning made that architecture reusable across tasks, scale unlocked emergent abilities, and RLHF made the result feel genuinely helpful to talk to. That chain of ideas - stretched across less than a decade - is the story of how we got from LSTMs to ChatGPT.